Odin — Technical Document: Klaviyo, Veeqo, Azure & Anchor Connect

Comprehensive reference for the recent phases. Everything here uses **real, connected values** — no "optional" anything. Where a value changes for production, it is listed in the **Production change board**.

- Document location: `docs/ODIN_PHASES_TECH_DOC.md` (this file) and `docs/ODIN_PHASES_TECH_DOC.pdf`.
- Companion: `ANCHOR_CONTRACT.md` (the interface contract) and `README_ANCHOR_AND_INTEGRATIONS.md`.

---

1. System overview

Three frontends, one brain, one anchor:

Component	Address (local now)	Role
Brain	`http://localhost:8000`	Local AI + dashboard API + agents (CTO→team); holds projects, chat (`/scope`), data.
Desktop app	Electron window → `:8000`	Original rich dashboard. Local only → no login.
Web app (Flutter)	`http://localhost:8080`	You, remotely. Mirrors desktop + Anchor. **Login required.**
Mobile app (Flutter iOS/Android)	device	Same app, mobile layout. **Login required** + Face ID first access.
Anchor (stand-in for the VM)	`http://localhost:9100`	Queue + auth + webhooks. The door third parties knock on.
Local poller	no port	Pulls tasks from the anchor over HTTP → runs CTO→team on the brain → posts results.
Third-party simulator	`third_party_sim.html` / `server/third_party.py`	An outside app sending a prompt.
Webhook receiver	temporary local page (see §11)	Where the third party *sees* the reply and acknowledges it.

End-to-end flow: `third party → anchor(9100) → local poller → brain(8000) CTO→team → real Klaviyo/Veeqo → result → anchor → webhook to the third party → third party ACKs (2xx) → request FINISHED`.

---

2. Klaviyo integration (Email)

- **Files:** `server/integrations/klaviyo.py`, `server/integrations/cred.py`.
- **Auth:** header `Authorization: Klaviyo-API-Key <KLAVIYO_API_KEY>` + `revision` header. Base `https://a.klaviyo.com/api`. Values from `oth/.cred`.
- **Capability:** generic `call(method, path, params, body)` reaches ANY Klaviyo endpoint; plus convenience wrappers (`lists`, `campaigns`, `flows`, `metrics`, …).
- **Read-only test fn:** `flows_summary()` → `{ok, total, by_status:{live, draft, …}}`.
- **Known real numbers (your account):** total **50** flows — **28 live / 22 draft**.
- **Used by:** the Email agent on `email`-type projects; the Anchor `process()` grounds answers in this live data.

3. Veeqo integration (Shipping / Fulfillment)

- **Files:** `server/integrations/veeqo.py`.

- **Auth:** header `x-api-key: <VEEQO_API_KEY>`. Base `https://api.veeqo.com`. Value from `oth/.cred`.
- **Capability:** generic `call(...)` + shipping wrappers (`orders`, `shipments`, `create_shipment`, `carriers`, `current_user`, …).
- **Read-only test fn:** `orders_summary(since_iso)` → `{ok, count, since}`.
- **Known real number (your account): 26** orders this week.
- **Used by:** the Shipping agent on `fulfillment`-type projects.

Read-only testing: all integration tests use GET only — they never create/modify anything in your Klaviyo/Veeqo accounts.

4. Azure VM

- **Control plane (`az`)** is used ONLY for firewall (NSG port-22 single-IP + ufw single-IP); ports **80/443 are sacred** (never touched). Build/rsync/reboot run as SSH user `mostafa`.
- **Deploy folders:** frontend → `~/organics`, backend → `~/bendcore` (never cross them).
- **Deploy is one step, non-recurring**, with a 10-round failure protocol that isolates the failed step, fixes, retries that step only — never re-runs reboot/reset.
- The VM is where the **Anchor backend + Web build** deploy in Phase 3 (see §13).

---

5. Anchor Connect — what it is

A stable external access layer so third parties and your own remote app reach local projects even though your machine has a dynamic IP behind NAT. The anchor holds a **smart queue**, authenticates callers, hands work to the local machine, and delivers **webhooks** with **ack + retry**.

Status lifecycle: `received → queued → sent_to_local → processing → response_received → webhook_sent → (acknowledged → finished)`; branches: `ack_pending` (awaiting retry), `failed` (retries exhausted), `cancelled`. **A request is FINISHED only after the local machine returned a result AND the webhook was acknowledged.** "Delete" is a final status flag, never a real delete; finished/failed are never re-polled.

Retry: `retry_count=4`, `retry_interval_sec=600` (configurable via `/anchor/v1/admin/config`).

6. Anchor — Phase 1 (queue + lifecycle)

- **Files:** `server/anchor_queue.py` (queue + lifecycle + webhook + retry, persisted in `data/anchor_queue.json`), `server/anchor.py` (in-process router `/anchor/v1` + `process()` = the real CTO→team orchestration grounded in real read-only data).
- **Tests:** `server/e2e_anchor.py`, `server/e2e_anchor_http.py`, `server/e2e_anchor_live.py`, runner `server/e2e_anchor_all.py`.

7. Anchor — Phase 2 (app + remote topology)

- **`odin_app/`** — Flutter app (one codebase: web + iOS + Android). Mirrors the desktop + Anchor page; shared `/scope` chat pool; New Project; device voice (TTS/STT).
- **`server/anchor_remote.py`** — the **stand-in for the deployed anchor** (real HTTP on `:9100`, same `/anchor/v1` contract, owns webhook delivery + retry sweep).
- **`server/local_poller_http.py`** — the local machine talking to the anchor **only over HTTP**.
- **`server/e2e_phase2.py`** — topology E2E (app ■ remote anchor ■ local).
- **`third_party_sim.html` / `server/third_party.py`** — the outside caller.
- **`server/gen_test_keys.py`** — generates/prints the auth keys.

8. Anchor — Phase 3 (planned, see §13)

Dockerize the anchor + systemd, TLS on the VM (443), serve the Flutter web build same-origin, HMAC-signed webhooks, per-device tokens. The local poller then points at the VM URL — **only the base URL changes**.

---

9. Authentication & security (REQUIRED — not optional)

Auth is real and saved now. Defaults for *verify email* and *2FA* are **off**, but the machinery is **fully functional** — turning them on in config makes them work. Nothing here is optional.

Seeded credentials (changeable in-app)

- **Email:** `tech@odinmarkets.com`
- **Password:** `12345678`
- Stored hashed in `oth/.auth` (gitignored). The owner can change the password in **Settings** → **Account** (web and Flutter).

Per-surface policy

Surface	Login	2FA	Verify email	Forgot password	Face ID
Desktop	none (local device)	—	—	—	—
Web	required	functional, default OFF	functional, default OFF	yes	—
Mobile (iOS/Android)	required	functional, default OFF	functional, default OFF	yes	**iPhone 13 Face ID = first access**

Machine-to-machine auth (always on when keys are set)

Link	Mechanism
Web/Mobile → Anchor (`/anchor/v1`)	`X-API-Key: <ANCHOR_API_KEY>` (required)
Web/Mobile → Brain (`/api/v1`)	`X-API-Key: <ODIN_API_KEY>` (required)
App → Brain native (`/projects`, `/scope`, …)	session token / `X-Odin-Token`
Local poller → Anchor	`X-API-Key: <ANCHOR_API_KEY>`
Anchor → third-party webhook	HMAC-signed payload (Phase 3)

Config flags (defaults, all functional)

`oth/auth_config.json`:

```
{ "require_login_web": true, "require_login_mobile": true,
  "twofa_enabled": false, "verify_email_enabled": false,
  "faceid_first_access": true, "desktop_open": true }
```

Real key values (local test — rotate before real exposure)

- `ANCHOR_API_KEY = od_anchor_h0awM4m6gZ0h_SCAvYppx6NYJZAOr1sv`
- `ODIN_API_KEY = od_odin_ss2dZSgUJGuriCvvfl63KGD2yBzjRbvf`
- `KLAVIYO_API_KEY`, `VEEQO_API_KEY` live in `oth/.cred` (not printed here).

---

10. API reference — Anchor on port 9100

Base: `http://localhost:9100` (local) → `https://<VM_IP_OR_DOMAIN>` (production). Header on every call: `X-API-Key: <ANCHOR_API_KEY>` (required), `Content-Type: application/json`.

```
GET  /anchor/v1/status
  200 {ok, service, auth_required}

POST /anchor/v1/requests                    # a third party submits work
  body {project, prompt, payload?, webhook_url, sender}
```

```
  200  {request_id, status:"queued", message}

GET  /anchor/v1/requests/{request_id}            # poll status + result
  200  {id, project, status, result, webhook_attempts, acked, error, created, updated, history}

POST /anchor/v1/requests/{request_id}/ack        # explicit acknowledgement
  200  {ok:true}

GET  /anchor/v1/poll                             # the LOCAL machine pulls work
  200  {tasks:[ {id, project, payload, webhook_url, ...} ]}

POST /anchor/v1/requests/{request_id}/result     # the LOCAL machine returns the result
  body {result}
  200  {ok:true}

GET  /anchor/v1/admin/requests?status=           # dashboard feed
GET  /anchor/v1/admin/counts
GET/POST /anchor/v1/admin/config                 # {retry_count, retry_interval_sec, poll_limit}

POST <webhook_url>                               # delivered TO the third party
  body {request_id, project, status:"completed", result, attempt}
  the third party MUST reply 2xx == acknowledgement; non-2xx -> anchor retries
```

---

11. The two REAL test sets

Common to both (required):

Field	Value
Anchor URL	`http://localhost:9100`
X-API-Key	`od_anchor_h0awM4m6gZ0h_SCAvYppx6NYJZAOr1sv`
Webhook URL	`http://localhost:9300/cb` (the webhook receiver page, §11.1)

Set A — Email / Klaviyo

Field	Value
Target project id	`email-klaviyo`
Prompt	`How many flows do we have, and how many are live vs draft?`
Expected	total **50** — **28 live / 22 draft**

Set B — Shipping / Veeqo

Field	Value
Target project id	`shipping-veeqo`
Prompt	`How many orders did we get this week?`
Expected	**26** orders this week

11.1 Webhook receiver (see send → reply → ack)

A temporary local page receives the anchor's webhook and lets you acknowledge it:

```
python3 server/webhook_receiver.py --port 9300     # serves /cb (POST) + a viewer at /
```

Open `http://localhost:9300/` to watch incoming webhook replies and click **Acknowledge** (reply 2xx). Use `http://localhost:9300/cb` as the `webhook_url` in the simulator.

---

12. Testing — where, how, what to expect

All test files live in `server/`. Run from the repo root.

12.1 One command — bring everything up

```
cd ~/Desktop/AiAgentWorkSpace/Odin-Orchestrator
./phase2_up.sh
```

Opens/starts: brain `:8000`, anchor `:9100`, local poller, desktop (Electron), web `:8080`, iOS simulator; prints the test keys. (`OPEN_DESKTOP=0 ./phase2_up.sh` to skip the desktop.)

12.2 Klaviyo/Veeqo + Anchor Phase 1

```
python3 server/e2e_anchor_all.py
```

Runs three suites and prints your real numbers:

```
Klaviyo flows: total=50  by status={live:28, draft:22}
Veeqo orders this week: 20
■ ALL ANCHOR E2E PASSED
```

Single suites: `e2e_anchor.py` (logic), `e2e_anchor_http.py` (HTTP+auth), `e2e_anchor_live.py` (real data via CTO→team).

12.3 Phase 2/3 topology

```
python3 server/e2e_phase2.py --smoke   # fast plumbing
python3 server/e2e_phase2.py           # REAL remote round-trip + real numbers
```

12.4 Multi-terminal manual test (what each terminal shows)

Terminal	Command	You should see
A — anchor	`python3 server/anchor_remote.py --port 9100`	`serving /anchor/v1 on http://0.0.0.0:9100`
B — poller	`python3 server/local_poller_http.py --anchor http://localhost:9100`	on a request: `processing <id> … via orchestration → result posted`
C — webhook	`python3 server/webhook_receiver.py --port 9300`	the delivered webhook body + an Acknowledge button at `:9300`
D — third party	`python3 server/third_party.py --anchor http://localhost:9100 --project email-klaviyo --prompt "How many flows are live vs draft?"`	`queued → processing → finished`, then the real answer (50/28/22)

Order: start A, B, C; then run D (or use `third_party_sim.html`). Watch B do the work, C receive the webhook, D print the final answer. The app's **Anchor** page (web/mobile) shows the same lifecycle.

12.5 The app

Open `http://localhost:8080` (web) or the iOS simulator. **Log in** (`tech@odinmarkets.com` / `12345678`). Settings → pick a profile, paste keys, Test connection. Create projects, chat (shared pool), try voice (■/■). The **bell** holds the docs (caching + this new doc).

---

13. Production change board

When the anchor is deployed to the Azure VM, change ONLY these. Everything else stays.

What	Local now	Production (VM)	Where to change
Anchor base URL	`http://localhost:9100`	`https://<VM_IP_or_domain>`	App **Settings** → **Deployed profile** → **Anchor base URL**; `local_poller_http.py --anchor`
Brain/API base URL	`http://localhost:8000`	(stays local; app reaches it via the anchor when abroad)	App Settings → API base URL
App access mode	`direct`	`anchor`	App Settings → mode
Transport	HTTP	**HTTPS/TLS on 443**	VM nginx/Caddy (443 is sacred/in use)
`ANCHOR_API_KEY`	test key	rotated strong key	`oth/.cred` + app Settings
`ODIN_API_KEY`	test key	rotated strong key	`oth/.cred` + app Settings
Webhook URL	`http://localhost:9300/cb`	the real caller's HTTPS endpoint	simulator / caller config
Project identifiers	`email-klaviyo`, `shipping-veeqo`, `organics`	the real deployed project ids	from `GET /projects` after creation
Webhook signing	none	**HMAC signature header**	anchor config

Deploy procedure (Phase 3): you create a `deployment` project in Odin, give it the deploy prompt (which I will provide), it runs the DevOps steps, you get the project id + the VM URL, then you set the values above in the app Settings and `oth/.cred`. The contract (`ANCHOR_CONTRACT.md`) is unchanged → seamless cutover.

---

14. In-app documentation (the bell)

The bell (web + Flutter + desktop) holds two styled doc cards: 1. **Caching & implementation** (existing). 2. **Klaviyo / Veeqo / Azure / Anchor** (this document) — new features, how the anchor works, the commands, the test steps, the file locations, and the production board.

Open the bell to read them in-app; the full source is this file + its PDF in `docs/`.